

Apache YARN

YARN was introduced in Hadoop 2.0.

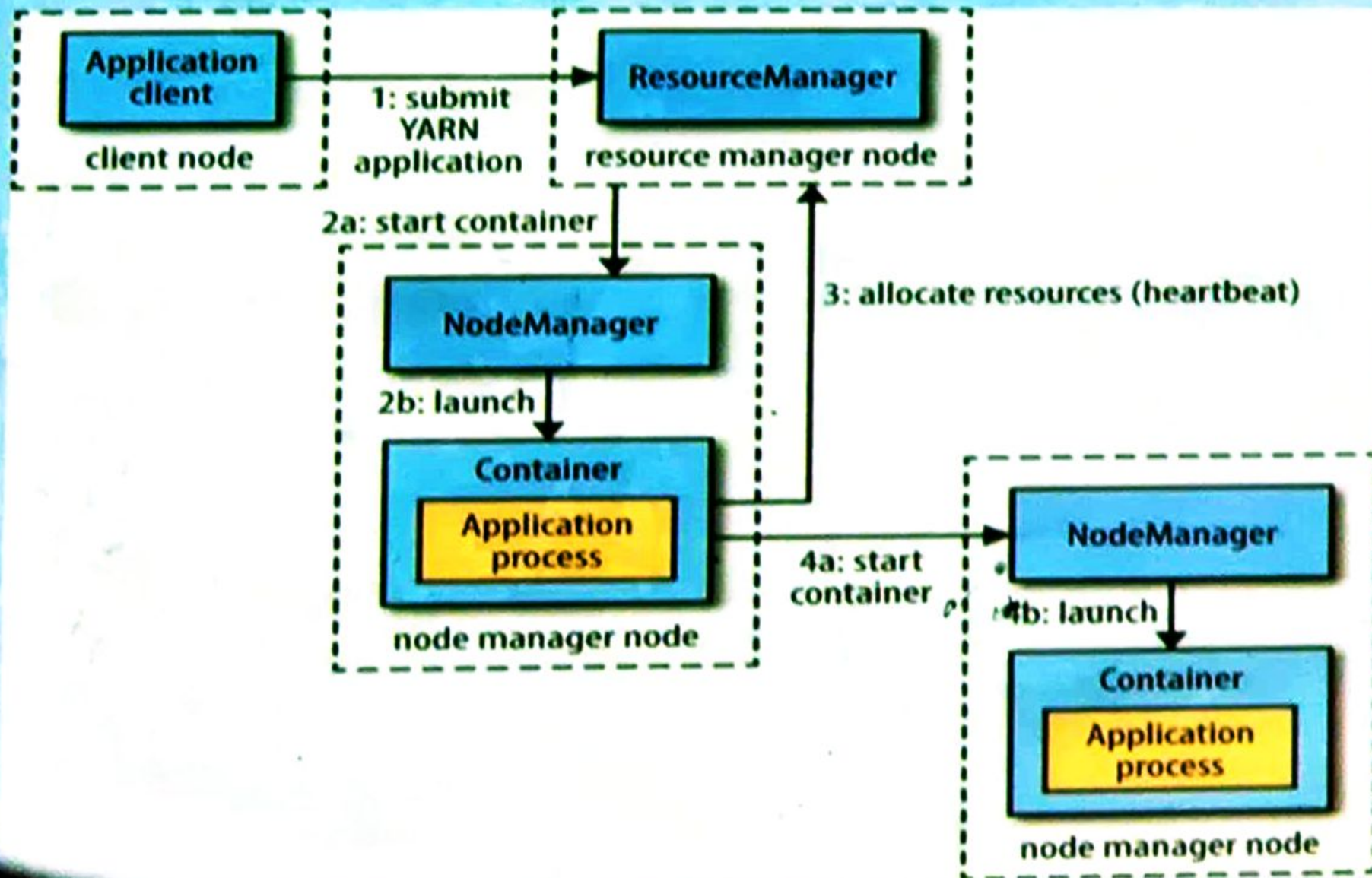
In Hadoop 1.0 a map-reduce job is run through a job tracker and multiple task trackers. Job of job tracker is to monitor the progress of map-reduce job, handle the resource allocation and scheduling etc. As single process is handling all these things, Hadoop 1.0 is not good with scaling. Also it makes Job tracker a single point of failure.

In 1.0, you can run only map-reduce jobs with hadoop but with YARN support in 2.0, you can run other jobs like streaming and graph processing.

In 1.0 slots are fixed for map and reduce tasks so while map is running you can't use reduce slots for map tasks because of that slots go waste, in 2.0 there is a concept of container, which has resources like memory and cpu-cores and any task can be run in it.



Architecture and working of Apache YARN





Apache YARN

Resource Manager:

It has two main components: Job Scheduler and Application Manager. Job of scheduler is allocated the resources with the given scheduling method and job of Application Manager is to monitor the progress of submitted application like map-reduce job. It has all the info of available resources.

Node Manager:

For each node there is a node manager running. It maintains the available resources on that particular node and notifies Resource Manager about the available resources when it starts. It launches the containers by providing the needed resources (memory, cpu etc.). These resources are allocated to container by Resource Manager. It manages the containers during its lifetime.



Apache YARN

Application Master:

It carries out the execution of job using different components of YARN. It is worked under Node Manager under the instructions of Resource Manager. One Application master is launched for each job.

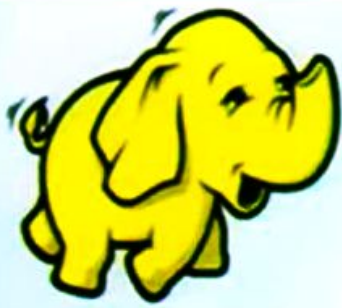
For resource allocation it talks to Resource Manager, for launching or stopping a container it talks to Node Manager.

It aggregates the status of task from different nodes and notifies the status of job to client as client polls on it.

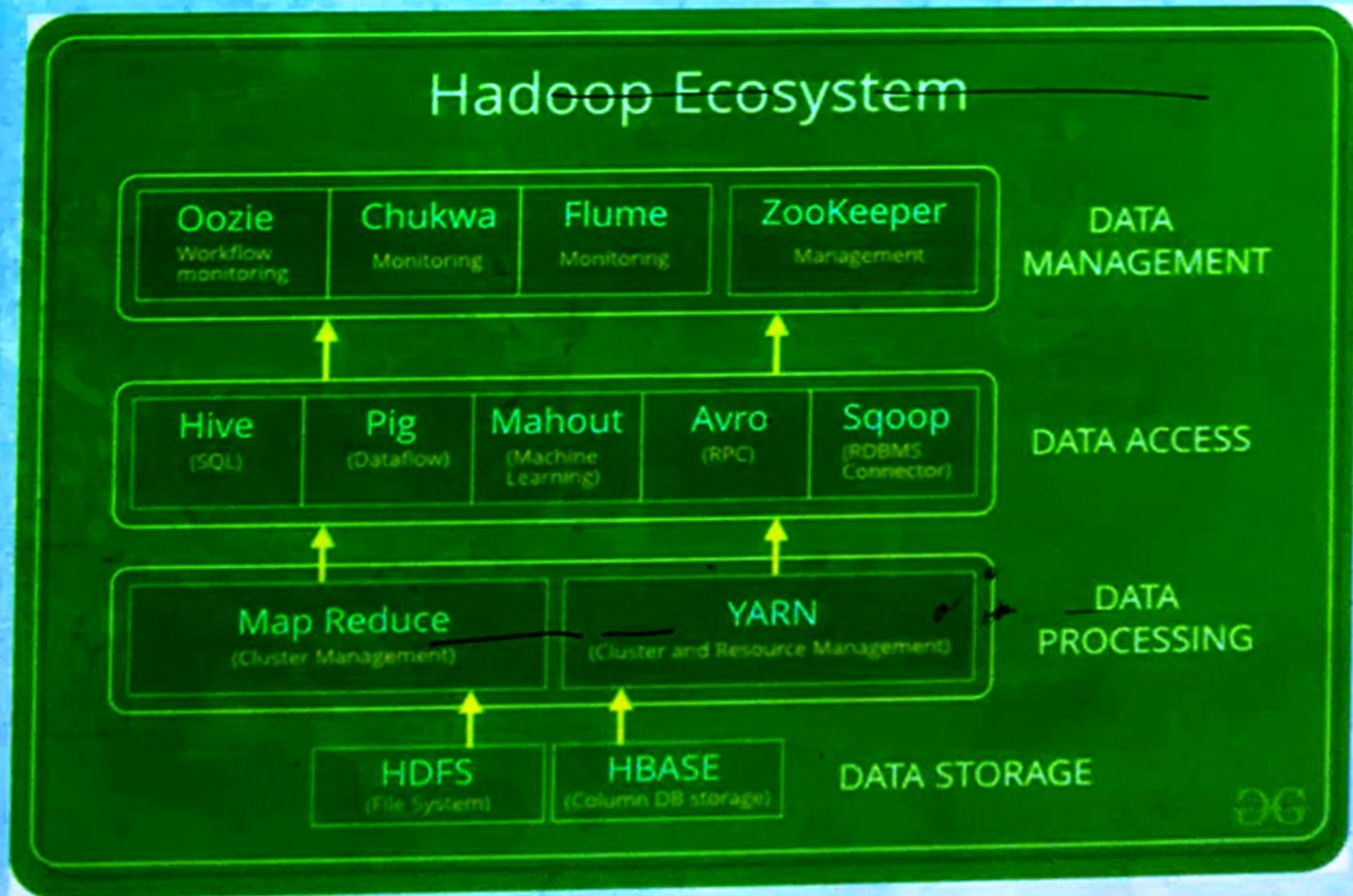
It also sends periodic heartbeat to Resource Manager to make sure Resource manager can launch a new Application Master in case of failure.

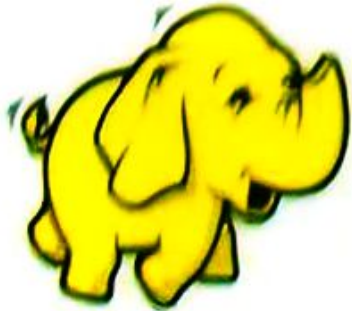
Container:

It is started by Node Manager. It consists of resources like memory, cpu core etc. For running a map or reduce task, Application Master asks Resource Manager for resources using which a container can be run.



Apache Hadoop Ecosystem



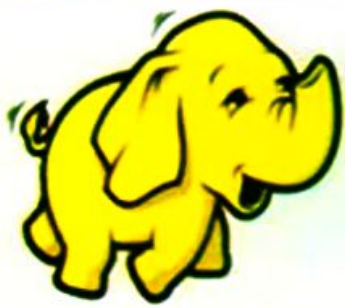


Data Management

Oozie:

Oozie simply performs the task of a scheduler, thus scheduling jobs and binding them together as a single unit. There are two kinds of jobs i.e. Oozie workflow and Oozie coordinator.

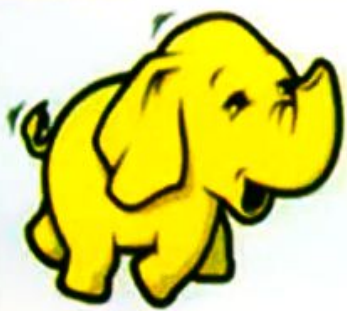
- Oozie workflow is the jobs that need to be executed in a sequentially ordered manner whereas Oozie Coordinator jobs are those that are triggered when some data or external stimulus is given to it.



Data Management

Chukwa:

- Apache Chukwa is an open source data collection system for monitoring large distributed systems.
- It is built on top of the HDFS and MapReduce framework and inherits Hadoop's scalability and robustness.
- It also includes a flexible and powerful toolkit for displaying, monitoring and analyzing results to make the best use of the collected data.

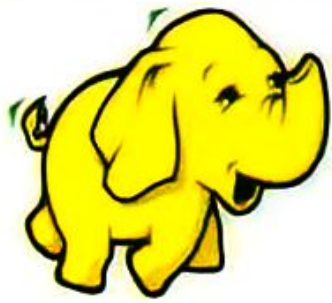


Data Management

FLUME: Apache Flume is a system used for moving massive quantities of streaming data into HDFS. Collecting log data present in log files from web servers and aggregating it in HDFS for analysis.

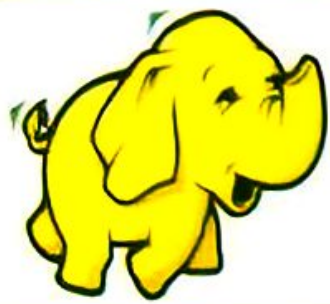
Flume supports multiple sources like –

- 'tail' (which pipes data from a local file and write into HDFS via Flume, similar to Unix command 'tail')
- System logs (Log Files)
- Apache log4j (enable Java applications to write events to files in HDFS via Flume).



Data Management

- **Zookeeper** is a distributed, open-source coordination service for distributed applications. It provides a way to ensure that nodes in a cluster are aware of each other and can coordinate their actions. It does this by maintaining a hierarchical tree of data nodes called “**Znodes**”, which can be used to store and retrieve data and maintain state information.
- ZooKeeper provides a set of primitives to implement higher-level services for synchronization, configuration and maintenance such as locks, barriers, and queues, that can be used to coordinate the actions of nodes in a distributed system.
- It is widely used in Hadoop, Kafka, and HBase, and it has become an essential component of many distributed applications.



Data Access

- **Hive:** Apache Hive is an open source data warehouse system for querying and analyzing large data sets that are principally stored in Hadoop files. It is commonly a part of compatible tools deployed as part of the software ecosystem based on the Hadoop framework for handling large data sets in a distributed computing environment.
- **Pig:** A high-level data-flow language and execution framework for parallel computation.
- **Mahout:** Mahout is open source framework for creating scalable machine learning algorithm and data mining library. Once data is stored in HDFS, mahout provides the data science tools to automatically find meaningful patterns in those big data sets.



Data Access

- **Avro:** It is an open source project that provides data serialization and data exchange services for Hadoop. These services can be used together or independently. Big data can exchange programs written in different languages using Avro.
- **Sqoop:** - It is a tool designed to transfer data between Hadoop and relational database servers. It is used to import data from relational databases such as MySQL, Oracle to Hadoop HDFS, and export from Hadoop file system to relational databases.



Data Storage

- **Hadoop Distributed File System (HDFS)**
- **HBase:** It is a distributed database that was designed to store structured data in tables that could have billions of row and millions of columns. HBase is scalable, distributed, and NoSQL database that is built on top of HDFS. HBase, provide real-time access to read or write data in HDFS.



Limitations of Hadoop

Hadoop can perform only batch processing, and data will be accessed only in a sequential manner. That means one has to search the entire dataset even for the simplest of jobs. A huge dataset when processed results in another huge data set, which should also be processed sequentially. At this point, a new solution is needed to access any point of data in a single unit of time (random access).